

LoRaWan Document

Setters:

1) void setDownStreamDataRate1()

a)Description:

Sets the downstream data rate for the first data rate option based on the upstream data rate and DLSettings.

b)Functionality:

-Computes the downstream data rate by subtracting DLSettings from the upstream data rate.

-If the resulting data rate is less than 0, it defaults to the lowest data rate (DR0).

-Sets the downstream data rate using the calculated or default value.

2)void setDownStreamDataRate2()

a)Description:

Sets the downstream data rate for the second data rate option based on the lower nibble of DLSettings.

b)Functionality:

- Extracts the lower nibble of DLSettings to determine the downstream data rate.

- Sets the downstream data rate using the extracted value.

3)void setDevAddr(const uint8_t d0, const uint8_t d1, const uint8_t d2, const uint8_t d3)

a)Description:

Sets the device address (DevAddr) for the end-device.

b)Parameters:

- d0, d1, d2, d3: Bytes representing the device address.

c)Functionality::

- Assigns the provided bytes to the devAddr array.

4)void setNetID(const uint8_t d0, const uint8_t d1, const uint8_t d2)

a)Description:

Sets the network identifier (NetID) for the end-device.

NetID is Network Server's unique identifier.

b)Parameters:

- d0, d1, d2: Bytes representing the network identifier.

c)Functionality:

- Assigns the provided bytes to the netID array.

5) void setNetSKey(const uint8_t n0, const uint8_t n1, const uint8_t n2, const uint8_t n3, const uint8_t n4, const uint8_t n5, const uint8_t n6, const uint8_t n7, const uint8_t n8, const uint8_t n9, const uint8_t n10, const uint8_t n11, const uint8_t n12, const uint8_t n13, const uint8_t n14, const uint8_t n15)

a)Description:

Sets the network session key (NetSKey) used for securing communication.

b)Parameters:

- n0 to n15: Bytes representing the network session key.

c)Functionality:

- Assigns the provided bytes to the netSKey array.

6)void setappKey(const uint8_t n0, const uint8_t n1, const uint8_t n2, const uint8_t n3, const uint8_t n4, const uint8_t n5, const uint8_t n6, const uint8_t n7, const uint8_t n8, const uint8_t n9, const uint8_t n10, const uint8_t n11, const uint8_t n12, const uint8_t n13, const uint8_t n14, const uint8_t n15)

a)Description:

Sets the application key (AppKey) used for encrypting join requests and responses.

b)Parameters:

- n0 to n15: Bytes representing the application key.

c)Functionality:

- Assigns the provided bytes to the appKey array.

7)void setnwKKey(const uint8_t n0, const uint8_t n1, const uint8_t n2, const uint8_t n3, const uint8_t n4, const uint8_t n5, const uint8_t n6, const uint8_t n7, const uint8_t n8, const uint8_t n9, const uint8_t n10, const uint8_t n11, const uint8_t n12, const uint8_t n13, const uint8_t n14, const uint8_t n15)

a)Description:

Sets the network key (NwkKey) used for securing communication between the end-device and the network server.

b)Parameters:

- n0 to n15: Bytes representing the network key.

c)Functionality:

- Assigns the provided bytes to the nwKKey array.

7)void setAppSKey(const uint8_t a0, const uint8_t a1, const uint8_t a2, const uint8_t a3, const uint8_t a4, const uint8_t a5, const uint8_t a6, const uint8_t a7, const uint8_t a8, const uint8_t a9, const uint8_t a10, const uint8_t a11, const uint8_t a12, const uint8_t a13, const uint8_t a14, const uint8_t a15)

a)Description:

Sets the application session key (AppSKey) used for securing application data.

b)Parameters:

- a0 to a15: Bytes representing the application session key.

c)Functionality:

- Assigns the provided bytes to the appSKey array.

7)void setCFList(const uint8_t a0, const uint8_t a1, const uint8_t a2, const uint8_t a3, const uint8_t a4, const uint8_t a5, const uint8_t a6, const uint8_t a7, const uint8_t a8, const uint8_t a9, const uint8_t a10, const uint8_t a11, const uint8_t a12, const uint8_t a13, const uint8_t a14, const uint8_t a15)

a)Description:

Sets the channel frequencies for communication based on the Channel Frequency List (CFList).

b)Parameters:

- a0 to a15: Bytes representing the channel frequencies.

c)Functionality:

- Extracts and assigns the provided bytes to the Channels and Channels_DL arrays, determining the frequencies for communication.

8) void setJoinEUI(const uint8_t a0, const uint8_t a1, const uint8_t a2, const uint8_t a3, const uint8_t a4, const uint8_t a5, const uint8_t a6, const uint8_t a7)

a)Description:

Sets the JoinEUI.

The JoinEUI is a global application ID in IEEE EUI64 address space that uniquely identifies the Join Server that is able to assist in the processing of the Join procedure and the session keys derivation.

b)Parameters:

- a0 to a7: Bytes representing the JoinEUI.

c)Functionality:

- Assigns the provided bytes to the joinEUI array.

9)void setDevEUI(const uint8_t a0, const uint8_t a1, const uint8_t a2, const uint8_t a3, const uint8_t a4, const uint8_t a5, const uint8_t a6, const uint8_t a7)

a)Description:

Sets the DevEUI.

The DevEUI is a global end-device ID in IEEE EUI64 address space that uniquely identifies the end-device. It is available on a device label, for device administration.

b)Parameters:

- a0 to a7: Bytes representing the DevEUI.

c)Functionality:

- Assigns the provided bytes to the devEUI array.

int16_t init():

a)Description:

Initialises the device by setting the frequency, transmission power, data rate, code rate, and some keys(JSIntKey,JSEncKey).

-JSIntKey is used to MIC Rejoin-Request type 1 messages and Join-Accept answers.

-JSEncKey is used to encrypt the Join-Accept triggered by a Rejoin-Request.

uint8_t join_request(uint8_t* const mpduPayload):

a)Description:

The Device sends this message in order to start up a connection with the server.

b)Parameters:

Takes a pointer as parameter that points to the array mpduPayload.

c)Functionality:

It initialises the pointer dataFrame that also points to mpduPayload.

Then fills AppEUI, the JoinEUI, devEUI and DevNonce. Indicating that the Mtype of the message is a join request.

-The AppEUI is a global application ID in IEEE EUI64 address space that uniquely identifies the entity able to process the JoinReq frame.

-DevNonce is counter that is set to zero at the first join request message sent by the device. it cannot be reused thus, it helps preventing attacks by replaying the message previously sent.

Finally, it calculates the message integrity code to ensure the message integrity when received.

bool join():

a)Description:

From an end-device's point of view, the join procedure consists of two MAC messages exchanged with the server, namely a join request and a join accept.

Returns true if the device has successfully joined the network.

b)Functionality:

- The method calls `defaultSetting()` that resets all parameters(`rxParamSetup`, channels available to be used, data rate range of each channel and channels' frequencies).

Assigns `LORA_FREQUENCY2` its default value. `//check`

- Initialise an empty mpdu payload array and calls the function `join_request(uint8_t* const mpduPayload)` as to send a join request message to the server.

- A random channel is selected from 16 channels, corresponding frequency and data rate are set.

- Waits for an accept message to be received after calling `setDownStreamDataRate1()` that sets a downlink data rate, and setting frequency that corresponds to the used channel.

- If the message is accepted the function returns a boolean true "accepted". Else tries another frequency (`LORA_FREQUENCY2`) and another data rate calling `setDownStreamDataRate2()` and waits to receive a message.

void send_message(uint8_t* const message, uint8_t len)

a)Descriptio*:

Sends a message from the end-device to the server over LoRaWAN. The function selects a random channel and checks its availability. It sets the frequency and data rate for transmission, then transmits the message. After transmission, it waits for a response and adjusts the downstream data rate and frequency if needed.

b)Parameters:

- message: Pointer to the message payload to be transmitted.
- len: Length of the message payload.

c)Functionality:

- Selects a random channel from the available ones.

- Sets the transmission frequency and data rate using the selected channel.

- Transmits the message via LoRaPhy.

- Waits for a response from the server within the specified RX delay.

- Adjusts downstream data rate and frequency after the first attempt.

- Retries transmission if the message is not acknowledged, up to the maximum number of retransmissions.